

Dynamic Coherent Cluster: A Scalable Sharing Set Management Approach

Julie Dumas[†], Eric Guthmuller[§], Frédéric Pétrot[‡]

^{†‡}Univ. Grenoble Alpes, CNRS, Grenoble INP*, TIMA, 38000 Grenoble, France

{julie.dumas, frederic.petrot}@univ-grenoble-alpes.fr

^{†§}Univ. Grenoble Alpes, CEA, LETI, 38000 Grenoble, France

eric.guthmuller@cea.fr

Abstract—The most widely used programming models expect hardware to guarantee coherent shared memory accesses. However, with the increasing number of integrated cores on chip, resource and performance efficient scalable cache coherence protocols are needed. To address the scalability issues due to the size of the sharing set, we propose to encode, on a fixed size bit-vector, a rectangular cluster whose goal is to cover most of the sharers. The cluster size is fixed but its height, width and position are determined for each cache block and can change during execution. We use a fixed size linked list for the first few outliers, and resort to broadcast when the list overflows. We compare our solution to snoop, directory-based full bit-vector, and Ackwise. It leads to similar mean latency and 10% less traffic than Ackwise, and only a few percent more than the complete sharing set on these metrics. More importantly, it generates ten times less broadcasts than Ackwise while using similar hardware resources for a 64 cores architecture.

I. INTRODUCTION

Manycore architectures can be roughly classified in two categories. First, non coherent clustered architectures with local memory at cluster level, mainly oriented towards specific applications on which real-time constraints can be taken into account. Applications running on these platforms are developed for high performance per watt ratio, and require a deep knowledge of the architecture for their developments [12]. Second, flat, CMP like, manycore architectures with several levels of cache in which memory is logically shared and coherent, even though it may be physically distributed. These manycore architectures aim at running legacy software, programmed using processes and threads, using typically posix-like APIs [16]. One of the sticking points of this type of architecture is the cache coherence protocol, that should scale with the number of caches in the system. There are two classes of cache coherence protocols [19]: snoop protocols that generate higher traffic but have a low hardware cost, and directory-based protocols that generate lower traffic but have a more important hardware cost because they must keep, for each cache block, a set of sharers in a directory. Our aim in this paper is to introduce a new sharing set management for cache coherence protocol and demonstrate that it is efficient in both resources and performances.

We have been guided in our approach by two observations. The first one is that although the number of core increases, the level of sharing of the data does not actually increase. Figure 1 (note the log scale on the y -axis) gives the averaged distribution of the number of sharers per cache block for 13 programs of the PARSEC [2] and SPLASH-2 [22] benchmarks executed on a 64 cores CMP architecture with write-back caches and MOESI (Modified, Owned, Exclusive, Shared, Invalid) cache coherence protocol. We can see that more than 93% of the blocks are shared by at most 8 cores. The outcome of this experiment is confirmed by the more formal study of [10] in which the authors show, using an analysis based on Amdahl's law, that the level of data sharing does not increase with the number of cores because the section of code that can run in parallel is fixed. The second observation is that modern SMP operating systems map the processes that share a lot of data on cores that are close to one another [7].

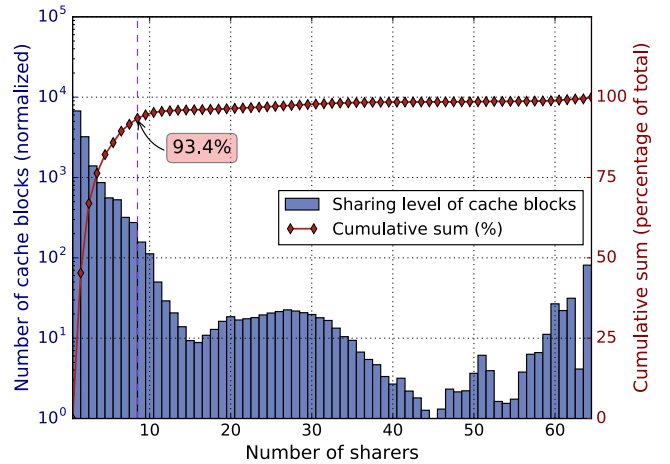


Fig. 1: Number of blocks shared by n cores on a 64 cores CMP

We assume an architecture with private L1 caches, L2 caches shared by all cores and a flat 2D mesh network-on-chip (NoC) interconnect. The contribution of this paper is the design and implementation of a hybrid sharing set management for cache coherence protocols. It relies on (a) a

*Institute of Engineering Univ. Grenoble Alpes

dynamic rectangular cluster intended to cover the majority of the sharers whose shape and position that can be efficiently encoded, (b) the use of a linked list for a small number of outliers, and (c) a broadcast mechanism when the linked list overflows. We call it Dynamic Coherent Cluster (DCC),

The remainder of the paper is organized as follows. Section II reviews the related work. Our DCC protocol concept is presented Section III and Section IV details its implementation. Section V presents the design of experiments while Section VI gives DCC performance evaluation. Finally, Section VII summarizes the results and concludes the paper.

II. RELATED WORK

Cache coherence protocols can be broadly classified in two classes: snoop and directory-based [19]. Snoop is appropriate for a small number of cores connected through a bus because it uses broadcasting, the natural type of transaction on a bus, and has no extra memory cost. However, with more cores this protocol is not scalable because first, it induces unnecessary coherence traffic, and second, large scale architectures are NoC based, and efficient broadcast on this kind of architecture is difficult. Directory-based protocols replace broadcast by multicast, by registering in a directory the list of caches that holds a given data at a given time. The initial proposal [4] is to use a full bit-vector in which one presence bit per processor is necessary. This straightforward solution requires a lot of memory to store only meta-data on large scale architectures, so different strategies to limit the directory size have been proposed. We briefly review them now.

A. Sparse directories

To avoid using full bit-vector to maintain the sharing set, sparse directories have been proposed. Among those proposals, the most well known one is Ackwise [14]. The idea is to limit the number of copies in the sharing set to a small number of sharers (less than 10) and resort to broadcast otherwise.

In [11], a coarse bit-vector directory scheme is proposed. When the number of sharers is too high, a full bit-vector is used in which each bit identifies a group of cores, multicast is then used to address each group of cores.

The authors of [10] propose to virtualize the CIDs: each memory page has its own mapping between virtual cores and physical cores. This mapping is sparse and the sharing set is thus encoded as a limited bit-vector. As in Ackwise, when the limit of virtual cores is reached for a page, broadcast is used to maintain cache coherence for this page.

B. Dynamic directories

Another class of directories allocates sharing sets in shared structures. This class is also sparse by nature and an imprecise mode must be used when the shared structure is full. The use of linked lists as dynamic sharing set, as proposed initially in [20], has led to a great deal of variations, e.g. Release Write-Through [15]. Linked list of Core IDs (CID) are allocated in a common heap and broadcast is used when the heap is full.

In [5], the authors use segments to encode only a subset of the bit-vector. These limited bit-vectors can be used in e.g. linked list directory approaches or coarse directory.

The authors of [17] extend this idea by mixing entries with several CIDs as in Ackwise and bit-vector segment entries allocated in a shared heap. If the block is private, the unique sharer CID is stored in place of the pointer to the heap. In [5], [17] segments are aligned respectively to K and 16 bits. An important point is that as opposed to these proposals, in DCC the cluster is neither fixed nor aligned, leaving more room for optimization.

SpongeDirectory is proposed by [23] and has two sharing set representations: one using CIDs and one using full bit-vector. These representations are stored in a common heap but contrary to [17], full bit-vectors span a fixed number of entries. When the number of sharers is low they use the list of CIDs, and switch to full bit-vector for broadly shared cache blocks. To free some space in the heap when needed, they invalidate other cache blocks.

III. DYNAMIC COHERENT CLUSTER CONCEPT

The main reason for which directory-based protocols may not scale is the size of the meta-data they need to keep track of shared data locations. According to the ITRS [1], in 10 years from now, micro-servers for data-centers will embed 128 cores, while more specialized manycore systems already integrate in the order of 256 cores [21], [6]. As the number of cores increases, broadcasting becomes impractical as it causes network congestion and suffers from high message latency. Additionally, at the same time, a full bit-vector meta-data to maintain the sharers of a given cache block takes more memory bits than the data itself. And finally, as noted in the introduction, the number of sharers of a data does not increase as the number of core does. Therefore, we propose DCC, a novel hybrid sharing set management strategy which takes benefits from these facts.

We assume, without loss of generality, that the N cores composing the system are placed on a 2D grid to ease the presentation of the concept. Our proposal main idea is to cover a, hopefully large, subset of the sharers using a rectangular shape of a fairly small area, and to encode the actual sharers in a bit-vector relative to this shape. This expectation is based on the two observations we made in the introduction: (a) a large majority ($\approx 93\%$ in our experimentations, see Figure 1) of cache blocks are shared by at most 8 sharers, and (b) operating systems tend to map sharers close to each other. We choose a rectangle to cover the nodes of the network because this shape is easy to encode, and, as opposed for example to a circle, is straightforward to map on a 2D mesh. Furthermore, the computation of the presence bits in the bit-vector is relatively simple, again as opposed to a circular or any more complex shape.

DCC needs the following resources:

- a per block bit-vector of size C to identify the sharers that belong to a rectangular cluster containing C cores,

- a per block variable of size k to define the origin (x, y) of the cluster inside the grid (k is a function of N), and a variable of size l to encode the shape of the cluster (l is a function of C),
- a heap of H elements shared among all directory entries to build a linked list of core identifiers,
- a per block pointer to a heap element.

For each cache block, the principle of DCC is as follows. First, cover as much sharer cores as possible with the rectangle by choosing its origin and form factor wisely. Second, build a list of outliers with the non covered cores and point to it. Third, if the length of the list of outliers exceeds a threshold T or if the heap is exhausted, indicate so by setting an overflow bit. Therefore, either the sharers are identified using the cluster bit-vector and the linked list, or all cores are considered sharers.

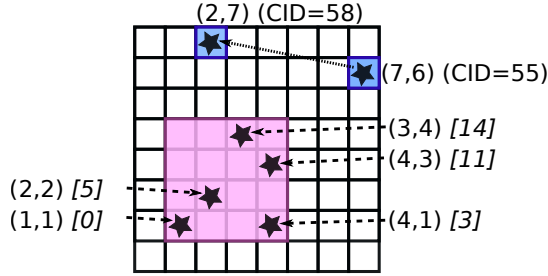


Fig. 2: DCC concept: stars represent sharers, the (x, y) coordinates are in the 2D mesh plan, the values between brackets represent the position of the core relative to the cluster

Figure 2 illustrates this idea with an example in which some cores belong to the cluster and two cores, CID 55 and 58, are in the linked list. Cores in the cluster have coordinates relative to the new origin encoded in the bit-vector relative to the shape of the cluster.

Theoretically, for a given block at a given time, covering as many sharers as possible with the cluster leads to an optimal use of resources and consequently to the best performances.

IV. DCC IMPLEMENTATION

A. Directory entry

We now focus on how to store all sharers using meta-data in the directory. In DCC, there are two modes: normal mode in which we use a cluster and a linked list, and broadcast mode. The former has a classical directory entry shown Figure 3 and a shared heap used to store the linked list [18].

field:	broadcast	origin	shape	bit vector	pointer
size:	1	$\log_2 N$	l	C	$\log_2 H$

Fig. 3: DCC directory entry

We give, for each bits field, its size in bit as a function of the number of cores N , the number of cores in the cluster C and the heap size H , assuming that these values are integral powers of 2. The number of bits l needed to encode the shape is the logarithm of the number of encompassing rectangles

with an area lower than C . For example, the different shapes for a cluster with 16 cores are 1×16 , 2×8 , 3×5 , 4×4 and the symmetric shapes. We enumerate 7 shapes, so we need 3 bits to encode the shape in this case. Note that the cache block state is left aside as it depends only on the implemented cache coherence protocol (MESI, MOESI, etc). When the broadcast bit is set, the origin field contains the *keeper* ID and the bit-vector and pointer fields are used to store the number of sharers. The *keeper* is the core that responds to each request for this block and because the number of sharers is known, only sharers have to answer to a broadcast request, greatly limiting the traffic. Regarding the heap, each element contains two items: a sharer ID ($\log_2 N$ bit) and a pointer to the next element ($\log_2 H$ bit).

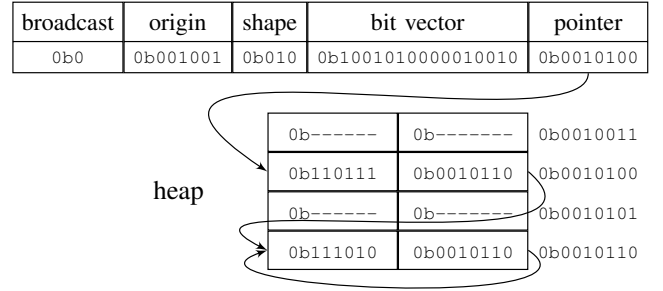


Fig. 4: DCC directory entry and heap content for the example

Figure 4 represents the directory entry and the heap content for the example of Figure 2. Here, we have a 64-cores platform, a cluster of size 16, and a heap with 128 entries. We can see that the cluster has the core 9 as origin, that its shape is 4×4 , and the bit-vector corresponds to the sharers in the cluster (the most significant bit corresponds to the first core in the cluster, it thus codes $\{0, 3, 5, 11, 14\}$). The pointer points to an element in the heap. The first element of the linked list stores the $CID = 55$ and a pointer to the next element in the heap. The second element contains the $CID = 58$ and a pointer to itself so as to signal the end of the list.

Given these values of N , C and H , the directory entry size is 33 bits to which we have to add the heap cost to evaluate the cost per directory entry. The heap costs $H \times (\log_2 H + \log_2 N) = 1664$ bits. Assuming one L2 cache contains 4096 blocks, the cost of the heap is $\frac{1664}{4096} \approx 0.41$ bit per block, so to speak insignificant.

B. Covering algorithms

We now present two relatively simple algorithms to dynamically choose the origin and shape of the rectangle to cover a subset of sharers. The goal of the paper being the presentation of the DCC principle, we do not focus here on elaborating a hardware feasible optimized sharers covering algorithm.

1) *DCC ideal*: This algorithm corresponds to the protocol performance upper bound. After each memory access (as it may lead to the addition or deletion of an element in the sharing set), we try, for all possible origins, all shapes of

maximum area and compute how many sharing cores it covers. We then select one cluster among the ones that cover the most cores. The upper bound is obtained with unlimited heap size ($H = \infty$) and no threshold for the linked list size ($T = N$), therefore all sharers are either in the cluster or in the linked list, and consequently there is no broadcast. A hardware implementation is difficult for two reasons. First, the unlimited heap size would require a number of elements corresponding to the product of the number of cores by the number of cache blocks. Second, as the cluster searching algorithm needs to respond in a few cycles as it stands on the critical path of the coherence protocol, it would require a huge amount of logic to perform the computation.

To have a tighter bound, we also consider DCC ideal with a limited heap and a threshold T to switch to broadcast mode. For example, with $T = 3$, if a core cannot be placed in the cluster and there are already 3 core IDs in the linked list, the broadcast mode is set. When this happens, all linked list elements are freed and the DCC directory entry is updated to contain the core ID of one of the cores in the cluster chosen to become the *keeper*, i.e. the core responsible for satisfying the coherence requests for this block. We also use this variant with an unlimited heap to determine an appropriate heap size for a specific threshold (see section V-B2).

Although this approach is useful to obtain an upper bound on the performance and to determine DCC parameters, we also propose an approach that is easy to implement in hardware for more realistic results.

2) *DCC first touch*: This *low cost* practical implementation of DCC uses a naïve placement of the cluster. The first access is placed once and for all in the cluster. At each new access, we try to change the origin and shape of the cluster so that it also includes the requester (the cluster somehow “grows” around the initial sharer). If there are no solutions, we try to add the new sharer in the linked list. If the length of the linked list is equal to T , there is no place for this new copy and we switch to broadcast mode. All copies are deleted from the linked list and from the cluster bit-vector, except for one which becomes the keeper.

This approach has a flaw because the first access is often done by the thread that initializes the data and launches the other threads, but does not quite use the data. In this case, we would typically like to put the initialization core in the linked list as an outlier.

V. DESIGN OF EXPERIMENTS

To benchmark the DCC variants, we first build a simulation environment and dimension the key parameters of DCC and reference cache coherence protocols parameters used for comparison.

A. Simulation environment

To evaluate our proposal we use the flow introduced in [8] and used in [9] to compare existing cache coherence protocols. It is summarized Figure 5. First, traces corresponding to the

(1) Simulate with gem5 and extract traffic (2) Trace based cache simulator (3) Analysis

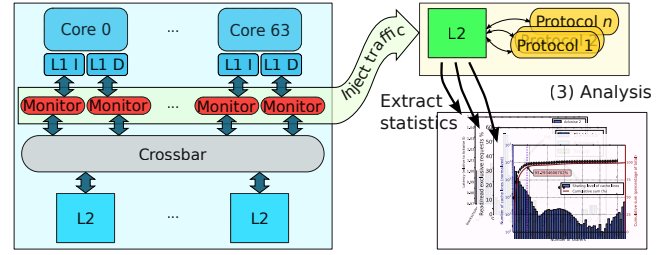


Fig. 5: Simulation flow in 3 steps

traffic between L1 caches and the network are extracted from gem5 [3] simulations. Second, these traces are injected in a high-level cache simulator that plays the different protocols using the traces. Third, from this simulation, we compute some metrics like latency, traffic or number of broadcasts.

The architecture simulated with gem5 is detailed Table I. The L2 is distributed: there are 64 L2 caches, shared by all cores.

TABLE I: gem5 parameters

CPU	64 cores Alpha 2GHz
L1 instructions	32kB, 2 way set associative
L1 data	64kB, 2 way set associative
Shared L2	256kB $\times$ 64, 8 way set associative
Block size	64B
Network	Full crossbar

Programs from the PARSEC [2] and SPLASH2 [22] benchmarks are executed on Linux in the simulation platform. The *simmedium* inputs are chosen to have relevant traces and we focus on the Region Of Interest. The ROI is the parallel compute section which induces that the initialization step is not present in our traces. gem5 is restarted at the beginning of the ROI and so caches start in cold state.

DCC relies on the fact that tasks that share data are placed on cores close to each other in a 2D mesh. However, gem5’s topology is a flat crossbar and Linux does not try to optimize task placement. So we perform an offline post-execution task placement optimization by considering a 2D mesh network topology. This optimization uses simulated annealing to place one task per core by minimizing the following energy function: $\sum_{i=0}^N \sum_{j=0}^N d_{ij} \times b_{ij}$, i.e. the sum for all pairs of cores of the distance d multiplied by the number of shared blocks b between these two cores. This static placement is used when we inject traces in our simulator.

We choose to focus on four sharing set schemes in addition to DCC variants: directory-based full bit-vector (named bit-vector in this paper), snoop, Ackwise and linked-list.

Bit-vector [4], [19]: the sharing set is stored in a bit-vector, each bit representing a core. So the bit-vector has a width equal to the number of cores in the system. The sharing set representation is perfect and thus it is our baseline.

Snoop [19]: this protocol is based on broadcast only and no meta-data is stored. Each request is broadcast to all cores and all cores answer.

Ackwise [14]: in this protocol, only k sharers are stored in the cache meta-data. Each sharer is represented by its CID and one is the *keeper* which responds to all read requests. If there are more than k sharers, the block switches to broadcast mode and we use only a counter of sharers.

Linked-list [18], [13]: in this case, a linked list is used to store CIDs. A first CID is stored in the meta-data, covering the immense majority of private cache blocks. Alongside this CID, a pointer is used to reference a linked-list stored in a heap. The heap is shared between all cache blocks of the L2 cache and list elements are allocated dynamically. The linked-list behaviour is similar to DCC: if the heap overflows or if the length of the linked-list stored in the heap has reached a threshold T , the cache block switches to broadcast mode. In broadcast mode, like in DCC, a counter of the number of sharers is stored instead of the pointer and the CID contains the *keeper* ID. Only the sharers have to answer to the broadcast, as the keeper knows when all responses have been provided.

Snoop and bit-vector are reference protocols with fixed sharing set representations. Ackwise and the linked-list protocols have dynamic sharing set implementations very similar in nature to DCC. Furthermore, they are well-known proposals from the state of the art. We believe that comparing DCC to these four protocols will give a fair view of its performances.

B. Parameters dimensioning

We now focus on DCC parameters exploration. Cluster size, linked list size and heap size are DCC's parameters. For each of them we explore different combinations with the aim of finding the best configuration.

1) *Cluster size C :* We first fix once and for all the cluster size. As we pay a relatively high cost for storing the cluster coordinates and shape, we would like to use a cluster size of 8 or 16 sharers. Figure 6 shows the average number of sharers put in the cluster for cache blocks that have at least 2 sharers and are not in broadcast mode. This experimentation uses the DCC ideal variant with an unlimited heap. Even though the cluster utilization rarely goes above 50%, we still see an interesting gain for a 16-wide cluster. Thus, in the rest of this paper, we use a cluster size C of 16 for all DCC variants.

2) *Linked list threshold T :* The linked list threshold constrains the maximum length of linked lists stored in the heap. This threshold is important for several reasons:

- It puts a hard limit on the time taken to traverse the list,
- It prevents highly shared cache blocks to trash the heap,
- Finally, a high number of copies in the linked list limits the chance of finding the best cluster.

To determine the value of this parameter, we run our experiment with an unlimited heap in DCC ideal for varying

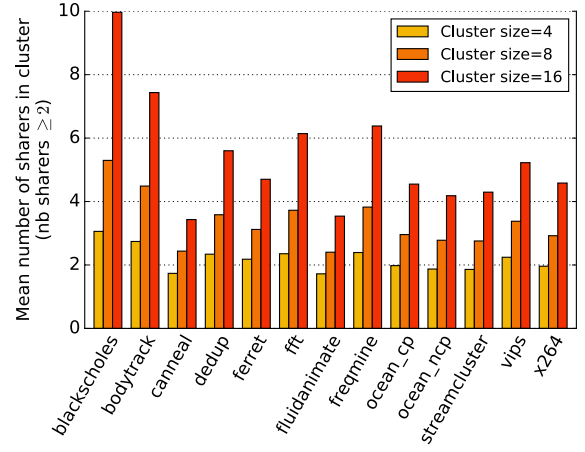


Fig. 6: Mean utilization of the cluster in DCC ideal for cache blocks having more than 2 sharers and not being in broadcast mode

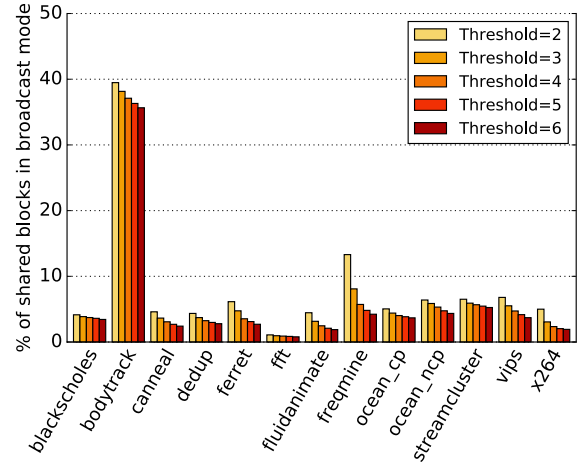


Fig. 7: Percentage of cache blocks in broadcast mode with an unlimited heap for DCC ideal

threshold values. Figure 7 presents the mean percentage of cache blocks that are in broadcast mode for all 10,000 cycles time windows and all L2 caches. In our experiments we focus on the mean percentage of cache blocks that are in broadcast mode. This percentage is greatly reduced up to a threshold of 4, that is at most 4 copies in the linked list. Past 4, the gains are less pronounced and the drawbacks may be too large. In the rest of this paper, we use a threshold T of 4 copies in the linked list for all DCC variants.

3) *Heap size H :* This parameter is correlated to the linked list threshold as a higher threshold will naturally lead to a higher utilization of the heap. Figure 8 shows the maximum, mean and quartiles (25th and 75th percentiles) heap utilization for all DCC variants. The rankings of DCC ideal and DCC first touch are not the same for each program. It can be explained by the different choices of cluster which induce variable linked list lengths. For all applications, the maximum heap

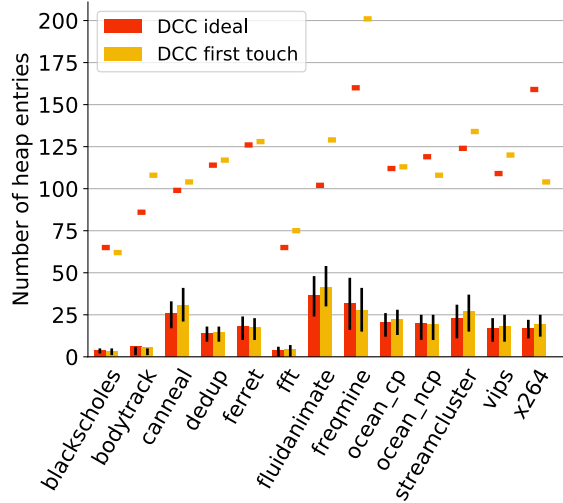


Fig. 8: Utilization of the heap structure (infinite heap size)

utilization rarely exceeds 128 elements. Moreover, the range of the quartiles shows that the distribution is concentrated around the mean, which never reaches 50 elements in the heap. Given this observation, we fix the heap size H to 128 elements.

C. Reference protocols parameters choice

Ackwise has a directory entry cost of $\log_2 N \times k + 1$ where N is the number of cores, k is the threshold and the last bit encodes broadcast mode. Section IV-A detailed the DCC entry and in our case the size is about 34 bits including the heap cost. So, we choose a threshold of 5 for Ackwise as its entry size is 31 bits for 64 cores. For more cores, the cost of Ackwise directory entry would grow faster than DCC one, assuming that heap and cluster sizes would not change much.

We fix the threshold to 5 for the linked-list protocol. This corresponds to 1 sharer in the directory entry and 4 in the linked list. Consequently, the access time is the same as DCC with a threshold of 4 but the directory entry size is smaller than DCC's. Indeed, the linked-list directory entry size is $\log_2 N + \log_2 H$ where N is the number of cores and H the heap size. To compare DCC to the linked-list protocol, we choose the same heap size. The directory entry size is thus 13 bits.

VI. DCC PERFORMANCE EVALUATION

This section focuses on the following metrics to compare our proposal with the state of the art: latency, traffic, number of broadcasts and directory entry size.

A. Latency

Figure 9 shows the latency between a L1 request and the moment the L1 receives the response. The snoop protocol is much worse for this metric with 43% more latency than bit-vector. Indeed, if a cache block has no sharer, the L2 cache must wait for responses from all L1 caches before sending its own data as a response. Thus it results in much higher latency in this case. We can see that the basic snoop protocol

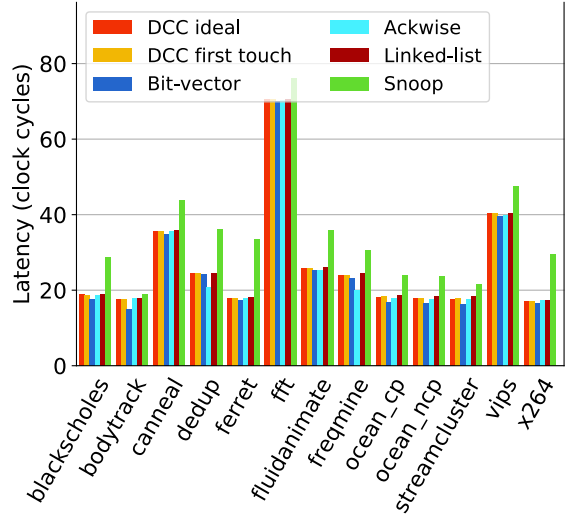


Fig. 9: Mean latency in clock cycles

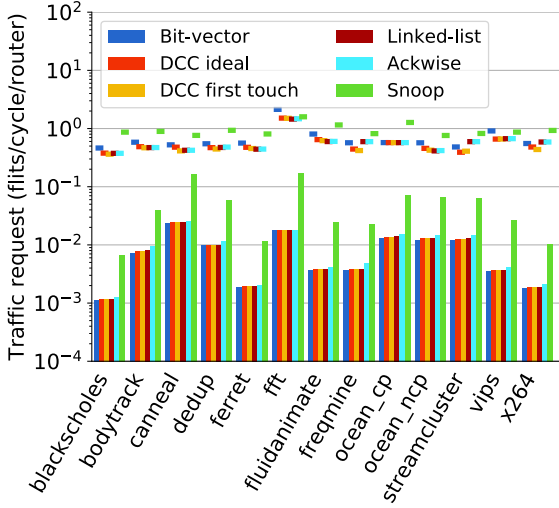
implementation is not suited for this kind of architecture. Ackwise and DCC have respectively 2% and 5-6% more latency than bit-vector on average. All these protocols use a keeper while in broadcast mode. So, the latency is mainly influenced by the way we choose *which* sharer is snooped in precise mode and which is the keeper in broadcast mode. However, as the task placement is optimized, the different choices performed by these protocols affect only marginally the final result.

B. Traffic

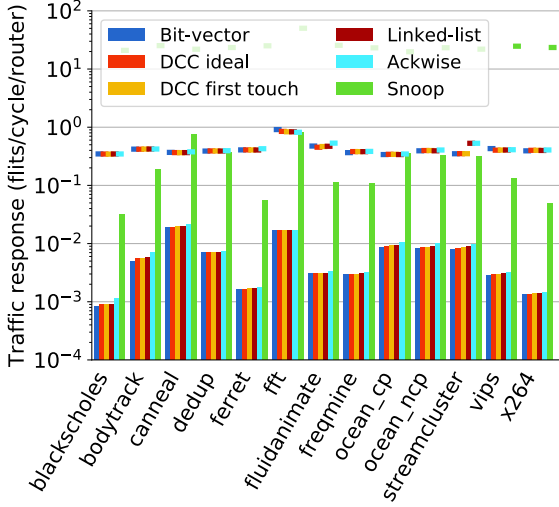
Figure 10a shows the average and maximum traffic in the request channel for all protocols and benchmarks. Figure 10b shows the same metric in the response channel. The DCC traffic is on average 4% higher for request and 5% higher for response channels than bit-vector. It is an important result as it shows how close our proposal is from the baseline bit-vector protocol. For this metric, DCC is better than Ackwise which produces 16% and 17% more traffic than bit-vector. Because the snoop protocol produces a lot of broadcast requests, inducing 64 responses each time, the NoC is literally flooded: it reaches a $40\times$ higher traffic level on the response channel. It is worth noting that our NoC model does not simulate contention, otherwise, the latency results for the snoop protocol would probably be much worse. Using such a protocol in a NoC-based 64 cores architecture would require highly efficient snoop filtering or a clever broadcast implementation.

C. Broadcast

DCC being based on a dynamic sharing set with a broadcast mode, measuring the number of broadcasts offers an accurate view of the efficiency of our proposal. Broadcasts consume a lot of bandwidth and power, and may limit performances while preventing scalability. Figure 11 shows the measure of the number of broadcasts for a reduced set of protocols. As



(a) Request channel



(b) Response channel

Fig. 10: Mean and max traffic for each protocol and each program

full bit-vector does not use broadcast, it does not appear in this figure. In this graph we can see that DCC ideal is on average one order of magnitude better than Ackwise, and two to three orders of magnitude better than snoop. It clearly shows that our proposal is able to more accurately map the sharing set than Ackwise with a similar hardware cost. Figure 12 focuses on the number of broadcasts for DCC first touch and linked-list normalized to DCC ideal. DCC first touch generates more broadcasts than DCC ideal but the number of broadcasts remains in the same order of magnitude. We also see that for the linked-list with $T = 5$, the number of broadcasts is on average twice that of DCC. This confirms that the cluster approach better represents the sharing set than the linked-list protocol and thus makes up for the higher hardware cost.

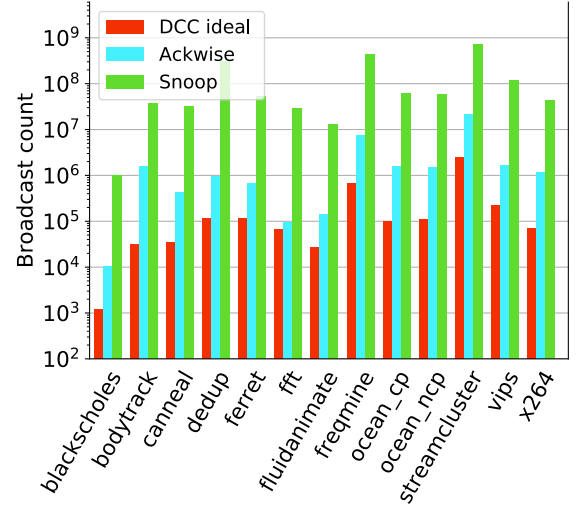


Fig. 11: Broadcast count

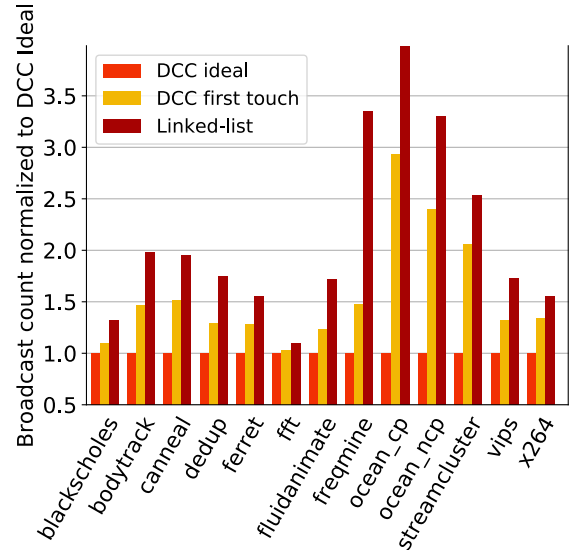


Fig. 12: Broadcast count normalized to DCC ideal

D. Comparison with other protocols

Table II summarizes the comparison between sharing set for three metrics : latency, traffic on both channel and number of broadcasts. Ackwise is the best protocol for the latency but for traffic and number of broadcasts our proposal, DCC, is better than Ackwise. The traffic produced by DCC first touch is close to the one produced by DCC ideal, but the number of broadcasts is quite larger with DCC first touch.

These results need to be evaluated also considering the sharing set hardware cost. Table III gives the number of bits of a directory entry for each sharing set management approach for architectures with 64, 128 and 1024 cores, all other things being equal. If we consider the 64 cores architecture, DCC has a hardware cost close to that of Ackwise. For the 1024 cores architecture, DCC hardware cost increases only slightly:

TABLE II: Sharing set comparison

Sharing set	Latency	Traffic request	Traffic response	Broadcast count
Snoop	+42.7%	$\times 6.32$	$\times 40.88$	$\times 789$
Bit-vector	+0% (Ref.)	+0% (Ref.)	+0% (Ref.)	No broadcast
Ackwise	+1.9%	+16.8%	+16.2%	$\times 12.67$
Linked list	+6.4%	+5%	+6.5%	+114%
DCC ideal	+4.8%	+3.9%	+4%	+0% (Ref.)
DCC 1st touch	+5.2%	+3.9%	+5.3%	+57.4%

TABLE III: Sharing set hardware cost

Sharing set management strategy	Directory entry size			
	Analytical formula	Cores		
		64	128	1024
Bit vector	N	64	128	1024
Ackwise	$\log_2(N) \times k + 1$	31	36	51
Linked List	$\log_2(N) + \log_2(H)$	13	14	17
DCC	$\log_2(N) + C + l + \log_2(H) + 1$	33	34	37

37 bits are necessary. On the contrary, Ackwise needs 51 bits on this architecture. The linked list is better from the hardware point of view but as our experiments show, DCC performs better than linked list for latency, traffic and number of broadcasts.

We therefore conclude that DCC is a good candidate for scaling: it has performances close to bit vector, which keeps a perfectly accurate set of sharers, while the number of bits of its directory entry increases slowly with the number of cores of the system.

VII. CONCLUSION

In this work, we have proposed DCC, a sharing set management strategy for directory-based cache coherence protocol. DCC uses a fixed size rectangular cluster whose origin and shape can be adjusted dynamically to cover as many sharer cores as possible. If some cores are not covered, a linked list allocated in a shared heap is used to store them. If the linked list or the heap overflows, broadcasting is used as a last resort. DCC exhibits a latency equivalent to Ackwise with 10% less traffic. It also has only 5% more latency and traffic than the full bit-vector representation. Moreover it generates a number of broadcasts one order of magnitude lower than Ackwise for similar hardware cost, which makes it particularly interesting for power sensitive applications or for high cores count architectures.

A direct follow-up of this work is the search for a wiser, but still acceptable in terms of hardware resources, sharers covering algorithm than DCC direct. Indeed, our experiments using DCC ideal prove that there is room for improvement on that topic.

REFERENCES

- [1] "The international technology roadmap for semiconductors, system drivers," 2015. [Online]. Available: www.itrs2.net/itrs-reports.html
- [2] C. Bienia, S. Kumar, J. P. Singh, and K. Li, "The parsec benchmark suite: Characterization and architectural implications," in *Parallel architectures and compilation techniques*. ACM, 2008.
- [3] N. Binkert, B. Beckmann, G. Black, S. K. Reinhardt, A. Saidi, A. Basu, J. Hestness, D. R. Hower, T. Krishna, S. Sadashti *et al.*, "The gem5 simulator," *ACM SIGARCH Computer Architecture News*, vol. 39, no. 2, pp. 1–7, 2011.
- [4] L. M. Censier and P. Feautrier, "A new solution to coherence problems in multicache systems," vol. c-20, no. 12, pp. 1112–1118, Nov. 1978.
- [5] J. H. Choi and K. H. Park, "Segment directory enhancing the limited directory cache coherence schemes," in *Symposium on Parallel and Distributed Processing*. IEEE, 1999, pp. 258–267.
- [6] B. D. de Dinechin, R. Ayrignac, P.-E. Beaucamps, P. Couvert, B. Ganne, P. G. de Massas, F. Jacquet, S. Jones, N. M. Chaisemartin, F. Riss *et al.*, "A clustered manycore processor architecture for embedded and accelerated applications," in *High Performance Extreme Computing Conference*, 2013, pp. 1–6.
- [7] M. Diener, E. H. M. Cruz, M. A. Z. Alves, P. O. A. Navaux, and I. Koren, "Affinity-based thread and data mapping in shared memory systems," *ACM Computing Surveys*, vol. 49, no. 4, p. 64, 2016.
- [8] J. Dumas, E. Guthmuller, C. F. Tortolero, and F. Pétrot, "A method for fast evaluation of sharing set management strategies in cache coherence protocols," in *International Conference on Architecture of Computing Systems*. Springer, 2017, pp. 111–123.
- [9] —, "Trace-driven exploration of sharing set management strategies for cache coherence in manycores," in *15th IEEE International New Circuits and Systems Conference*. IEEE, 2017, pp. 77–80.
- [10] Y. Fu, T. M. Nguyen, and D. Wentzlaff, "Coherence domain restriction on large scale systems," in *International Symposium on Microarchitecture*. ACM, 2015, pp. 686–698.
- [11] A. Gupta, W.-D. Weber, and T. Mowry, "Reducing memory and traffic requirements for scalable directory-based cache coherence schemes," in *Scalable shared memory multiprocessors*. Springer, 1992, pp. 167–192.
- [12] J. Hascoët, K. Desnos, J.-F. Nezan, and B. D. de Dinechin, "Hierarchical dataflow model for efficient programming of clustered manycore processors," in *28th International Conference on Application-specific Systems, Architectures and Processors*. IEEE, 2017, pp. 137–142.
- [13] J. Jones and R. L. Haggard, "Hardware-based methods for maintaining cache coherency in a multiprocessor system," in *Proceedings of Thirtieth Southeastern Symposium on System Theory*, Mar 1998, pp. 549–553.
- [14] G. Kurian, J. E. Miller, J. Psota, J. Eastep, J. Liu, J. Michel, L. C. Kimerling, and A. Agarwal, "Atac: a 1000-core cache-coherent processor with on-chip optical network," in *Parallel architectures and compilation techniques*. ACM, 2010, pp. 477–488.
- [15] H. Liu, C. Devigne, L. Garcia, Q. Meunier, F. Wajsburt, and A. Greiner, "Rwt: Suppressing write-through cost when coherence is not needed," in *IEEE Computer Society Annual Symposium on VLSI*. IEEE, 2015, pp. 434–439.
- [16] M. M. Martin, M. D. Hill, and D. J. Sorin, "Why on-chip cache coherence is here to stay," *Communications of the ACM*, vol. 55, no. 7, pp. 78–89, 2012.
- [17] S. Shukla and M. Chaudhuri, "Pool directory: Efficient coherence tracking with dynamic directory allocation in many-core systems," in *33rd IEEE International Conference on Computer Design*. IEEE, 2015, pp. 557–564.
- [18] R. Simoni and M. Horowitz, "Dynamic pointer allocation for scalable cache coherence directories," in *International Symposium on Shared Memory Multiprocessing*. IPS Press, 1991, pp. 72–81.
- [19] D. Sorin, M. Hill, and D. Wood, "A primer on memory consistency and cache coherence," *Synthesis Lectures on Computer Architecture*, vol. 16, 2011.
- [20] M. Thapar, B. Delagi, and M. J. Flynn, "Linked list cache coherence for scalable shared memory multiprocessors," in *Proceedings of Seventh International Parallel Processing Symposium*. IEEE, 1993, pp. 34–43.
- [21] D. Wentzlaff, P. Griffin, H. Hoffmann, L. Bao, B. Edwards, C. Ramey, M. Mattina, C.-C. Miao, J. F. Brown III, and A. Agarwal, "On-chip interconnection architecture of the tile processor," *IEEE micro*, vol. 27, no. 5, pp. 15–31, 2007.
- [22] S. C. Woo, M. Ohara, E. Torrie, J. P. Singh, and A. Gupta, "The splash-2 programs: Characterization and methodological considerations," in *ACM SIGARCH computer architecture news*, vol. 23, no. 2, 1995, pp. 24–36.
- [23] L. Zhang, D. Strukov, H. Saadeldien, D. Fan, M. Zhang, and D. Franklin, "Spongedirectory: Flexible sparse directories utilizing multi-level memories," in *Parallel architectures and compilation techniques*. ACM, 2014, pp. 61–74.